

Gazebo Classic 11.15.1: Kapsamlı Teknik Analiz ve Yetenek Raporu

BÖLÜM 1: TEMEL MİMARİ VE FELSEFE

1.1 Giriş: Gazebo Classic 11.15.1 Ekosistem Analizi

Bu rapor, Gazebo simülatörünün 11.15.1 sürümünün, yani "Gazebo Classic" olarak bilinen ve Ocak 2025'te Kullanım Ömrü Sonu'na (End-of-Life, EOL) ulaşacak olan son Uzun Süreli Destek (LTS) sürümünün, kapsamlı bir teknik analizini sunmaktadır. Gazebo, robotik algoritmaları test etmek, robot tasarımlarını doğrulamak ve yapay zeka sistemlerini karmaşık iç ve dış mekan ortamlarında gerçekçi senaryolar kullanarak eğitmek için kullanılan güçlü bir 3D dinamik simülatördür.

Bu belge, Gazebo Classic 11'in mimari temelinden fizik motorlarına, sensör modellemesinden Robot Operating System (ROS) ile entegrasyonuna kadar her bir bileşenin "neler yapabildiğini" ve daha da önemlisi, bunları "nasıl yaptığını" derinlemesine inceleyen bir referans dokümanı niteliğindedir. Bu sürüm artık yerini modern Gazebo (önceki adıyla Ignition) platformuna bırakmış olsa da, bu rapor, mevcut sistemlerin bakımını yapan veya bu platformun mirasını anlamak isteyen geliştiriciler için son derece ayrıntılı bir teknik döküm sunmayı amaçlamaktadır.

1.2 Temel Mimari: gzserver (Sunucu) ve gzclient (İstemci) Ayrımı

Gazebo'nun gücünün temelinde, simülasyonun çekirdek işlevlerini görselleştirmeden ayıran dağıtık mimarisi yatar. Bu ayrım, `gzserver` ve `gzclient` olmak üzere iki temel yürütülebilir program aracılığıyla gerçekleştirilir.

gzserver (Sunucu): Simülasyonun Beyni `gzserver`, simülasyonun tüm "ağır iş yükünü" gerçekleştiren çekirdek süreçtir. Görevleri şunlardır:

- Fizik Simülasyonu:** Open Dynamics Engine (ODE), Bullet, Simbody veya DART gibi seçilen fizik motorunu çalıştırır ve tüm dinamik hesaplamaları (çarpışmalar, yerçekimi, eklem kuvvetleri) yapar.
- Sensör Veri Üretimi:** Kameralar, Lidar'lar (ray sensors) ve IMU'lar gibi tüm sensörlerin verilerini üretir.
- Dünya Durumu Yönetimi:** Simülasyondaki her modelin pozunu (pose), hızını ve diğer durumlarını (state) günceller.
- Arayüzsüz (Headless) Çalışma:** `gzserver`, herhangi bir grafik arayüzü olmadan çalışabilir. Bu, simülasyonların bulut sunucularında, sürekli entegrasyon (CI/CD) sistemlerinde veya grafik kartı olmayan güçlü işlemcilerde çalıştırılması için hayati öneme sahiptir.

gzclient (İstemci): Simülasyonun Gözleri `gzclient`, çalışan bir `gzserver` sürecine bağlanan ve kullanıcıya simülasyonu görselleştiren grafik arayüzdür.

- Görselleştirme:** Simülasyon durumunu `gzserver` 'dan alır ve 3D ortamı (OGRE render motorunu kullanarak) render eder.
- Kullanıcı Etkileşimi:** Kullanıcının simülasyonla etkileşime girmesini sağlar. Bu, modelleri fare ile taşımak, onlara kuvvet uygulamak veya eklemeleri manuel olarak hareket ettirmek gibi eylemleri içerir.
- Durum:** `gzclient`, `gzserver` olmadan anlamsızdır; yalnızca çalışan bir simülasyon sunucusunun "istemcisi" olarak işlev görür.

`gazebo` komutunun kendisi, bu iki süreci (`gzserver` ve `gzclient`) tek bir komutla başlatan ve birbirine

bağlayan bir kolaylaştırıcıdır.

1.3 Dahili İletişim: Gazebo Transport Katmanı (Gazebo Transport)

`gzserver` ve `gzclient` arasındaki bu ayrım, güçlü ve esnek bir iletişim katmanı gerektirir. Gazebo, bu iletişimi kendi özel "Transport Library" (Taşıma Kütüphanesi) üzerinden sağlar.

Nasıl Çalışır: Gazebo'nun taşıma katmanı, TCP/IP soketleri üzerinden çalışan gelişmiş bir mesajlaşma sistemidir. Bu sistemin temelinde şu teknolojiler yatar:

1. **Boost ASIO:** Platformlar arası asenkron G/Ç (I/O) işlemleri ve ağ iletişimi için kullanılır.
2. **Google Protobufs (Protocol Buffers):** Mesajların serileştirilmesi (serialization) için kullanılır. Protobufs, veriyi platformdan ve dilden bağımsız, verimli bir ikili (binary) formata dönüştürür.

Bu mimari, **Yayınla/Abone Ol (Publish/Subscribe)** modelini kullanır:

- **Topic (Konu):** Mesajların gönderildiği, isimlendirilmiş bir kanaldır (örn. `~/world_stats`).
- **Publisher (Yayıncı):** Bir topic'e, belirli bir Protobuf mesajı formatında (örn. `msgs::WorldStatistics`) veri yayınlayan bir bileşendir (örn. `gzserver`).
- **Subscriber (Abone):** Bir topic'e abone olan ve her yeni mesaj geldiğinde bir "geri çağırma" (callback) fonksiyonu tetiklenen bir bileşendir (örn. `gzclient`).

Bu mimarinin `gzserver` / `gzclient` ayrımıyla birleşimi, Gazebo Classic 11'in en güçlü yönüdür. `gzserver` bir "simülasyon API sunucusu" gibi davranır. `gzclient`, C++ ile yazılmış "varsayılan" istemcidir. Ancak, Protobuf mesajlarını TCP/IP üzerinden gönderebilen *herhangi bir uygulama* da bir istemci olabilir:

- **Gzweb**, Gazebo Transport mesajlarını WebGL'e çeviren bir NodeJS köprüsü kullanan bir "web istemcisidir".
- **GazeboJS**, JavaScript/NodeJS'in doğrudan Gazebo topic'lerine abone olmasına veya yayın yapmasına izin veren bir "NodeJS istemcisidir".
- `gazebo_ros_pkgs` (Bölüm 7'de detaylandırılacaktır), Gazebo Protobuf mesajlarını ROS 1 mesajlarına ve servislerine çeviren bir "ROS köprüsüdür".

Sonuç olarak, Gazebo Classic 11'in mimarisi, simülasyonun tek bir monolitik program olmaktan çıkıp, çok dilli bir ekosisteme hizmet veren, esnek ve güçlü bir sunucu-istemci yapısına dönüşmesini sağlamıştır.

BÖLÜM 2: GÖRSELLEŞTİRME VE ÇEVRE (RENDERING)

2.1 Render Motoru: OGRE (Object-Oriented Graphics Rendering Engine)

Gazebo Classic 11, tüm 3D görselleştirme görevleri için **OGRE** (Object-Oriented Graphics Rendering Engine) grafik motorunu kullanır. Render kütüphanesinin çift amacı vardır:

1. **Kullanıcı Geri Bildirimi:** `gzclient` 'ta kullanıcıya yüksek kaliteli aydınlatma, gölgeler ve dokular içeren gerçekçi bir 3D ortam sunmak.
2. **Sensör Veri Üretimi:** Simüle edilmiş sensörler, özellikle de GPU tabanlı sensörler (kameralar, derinlik kameraları, GPU Lidar'lar) için piksel ve derinlik verisi üretmek.

2.2 Görsel Yetenekler ve Ayarlar

OGRE motoru, standart aydınlatma ve gölgelerin ötesinde gelişmiş görsel efektler sağlar:

- **Lightmaps:** Daha gerçekçi ve verimli aydınlatma için modellere önceden hesaplanmış ışık haritaları uygulanabilir.

- **Stereo Görüntü:** Simülasyonun, 3D stereo gözlüklerle (NVIDIA 3D Vision gibi) kullanılmak üzere stereo görüntüler oluşturmaya sağlanabilir.
- **VR Entegrasyonu:** Gazebo, Oculus Rift gibi Sanal Gerçeklik (VR) başlıklarıyla doğrudan entegrasyonu destekler.
- **Arazi Optimizasyonu (Paging):** Büyük arazilerle çalışırken, render motoru, kameranın uzaklığına bağlı olarak arazi parçalarını (subterrains) dinamik olarak yükleyip kaldırarak performansı optimize edebilir.

2.3 Aktörler (Actors): Fizik Dışı Animasyon

Simülasyonlarda "canlı" bir dünya yaratmak (örn. otonom bir aracın test edildiği kalabalık bir şehir) muazzam bir hesaplama yükü getirir. Eğer şehirdeki her bir yaya veya araç, tam bir fiziksel model (Bölüm 3) olsaydı, yüzlerce varlığın çarpışma ve dinamik hesaplamaları (Bölüm 4), simülasyonu çalışamaz hale getirirdi.

Gazebo bu sorunu "Aktörler" (Actors) adı verilen zekice bir konseptle çözer. Bir aktör, fiziğe tabi *olmayan* (yerçekiminden veya çarpışmalardan etkilenmeyen), ancak önceden tanımlanmış yolları takip eden animasyonlu bir varlıktır.

Aktörler, fizik motorunu (CPU) meşgul etmezler, ancak render motoru (GPU) tarafından görselleştirilirler ve sensörler (kamera, Lidar) tarafından algılanabilirler. Bu, simülasyonu yavaşlatmadan, görsel ve algısal olarak zengin bir ortam yaratmanın tek yoludur.

İki tür aktör animasyonu desteklenir:

1. **İskelet Animasyonu (Skeleton Animation):** Bir model içindeki linkler arasındaki göreceli hareketler (örn. bir insanın yürüme döngüsü). Bu animasyonlar COLLADA (.dae) veya BVH (.bvh) dosyalarından içe aktarılabilir.
2. **Senaryolu Yörüngeler (Scripted Trajectories):** SDF dosyası içinde `<script>` etiketiyle tanımlanan, aktörün belirli zamanlarda belirli pozlara ulaşmasını sağlayan bir yörüngedir (örn. bir aracın bir rota üzerinde gezinmesi).

Bu iki tür birleştirilebilir; örneğin, belirli bir yörüngeyi (scripted trajectory) takip ederken aynı zamanda yürüme animasyonunu (skeleton animation) oynatan bir insan modeli oluşturulabilir.

BÖLÜM 3: DÜNYA VE ROBOT MODELLEME (SDF)

3.1 Simulation Description Format (SDF) ile Dünya Tanımlama

Gazebo'daki her şey, SDF (Simulation Description Format) adı verilen XML tabanlı bir formatla tanımlanır. SDF, sadece robot modellerini değil, aynı zamanda tüm simülasyon dünyasını (ışıklar, fizik, ortam) tanımlamak için de kullanılır.

Bir `<world>` SDF dosyası oluştururken, en kritik performans optimizasyonlarından biri **statik** ve **dinamik** varlıkları doğru tanımlamaktır:

- **Statik Varlıklar (`<static>true</static>`):** Bunlar, simülasyon sırasında asla hareket etmeyecek olan varlıklardır (örn. zemin düzlemi, binalar, duvarlar). Fizik motoru bu varlıkları tamamen yok sayar, bu da hesaplama yükünü sıfıra indirir.
- **Dinamik Varlıklar (`<static>>false</static>` veya etiket yok):** Bunlar, fizik motoru tarafından yönetilen, ataleti olan, yerçekiminden etkilenen ve çarpışmalara tepki veren varlıklardır (örn. robotlar, toplar, sandalyeler).

3.2 Gelişmiş Arazi: Dijital Yükselik Modelleri (DEM)

Düz bir zemin düzlemi (ground_plane) yerine gerçek dünyadan alınmış karmaşık araziler oluşturmak için Gazebo, Dijital Yükseklik Modellerini (DEM) destekler.

Bu sürecin adımları şöyledir:

1. **Gereksinimler:** Öncelikle, coğrafi verileri işlemek için kullanılan GDAL (Geospatial Data Abstraction Library) kütüphanelerinin sisteme kurulması gerekir.
2. **Veri Hazırlama:** Bir DEM dosyası (örn. `mtsthelens.dem`) temin edilir. Bu dosyalar genellikle çok yüksek çözünürlüklü olduğundan, Gazebo'da verimli çalışması için `gdalwarp` aracıyla yeniden ölçeklendirilmesi önerilir (örn. `gdalwarp -ts 129 129...`).
3. **SDF Entegrasyonu:** DEM dosyası, SDF'de bir `<heightmap>` geometrisi olarak tanımlanır. Bu geometri hem `<collision>` (çarpışma) hem de `<visual>` (görsel) etiketleri içinde kullanılır.
 - `<uri>` : Ölçeklendirilmiş `.dem` dosyasının yolunu belirtir.
 - `<size>` : Yükseklik haritasının simülasyon içinde alacağı gerçek X, Y ve Z (yükseklik ölçeği) boyutlarını metre cinsinden tanımlar.
 - `<pos>` : Genellikle DEM'in minimum yüksekliğini sıfırlamak için bir Z-ofseti ayarlamak amacıyla kullanılır.

3.3 Robot Modellemenin Anatomisi (SDF ve URDF)

Gazebo'nun "incik cıncığına kadar" olan ayarlarının büyük çoğunluğu, robotun veya modelin SDF dosyasında yatar. Gazebo, ROS (Robot Operating System) standardı olan URDF (Unified Robot Description Format) formatını da desteklese de, bu, temel bir yanılgıya yol açmamalıdır: **Gazebo'nun yerel formatı SDF'dir.**

Bir URDF dosyası `gazebo_ros_pkgs` (Bölüm 7) tarafından yüklendiğinde, anında SDF formatına dönüştürülür. URDF'nin desteklemediği tüm gelişmiş simülasyon parametreleri (fizik, sensör gürültüsü, eklentiler), URDF dosyası içine gömülen özel `<gazebo>` etiketleri aracılığıyla eklenir. Bu nedenle, gelişmiş bir simülasyon yapmak, özünde SDF yazmayı öğrenmek demektir.

Bir `<model>` SDF dosyası, bir robotu oluşturan şu temel yapı taşlarından oluşur:

- `<link>` (**Bağlantı**): Robotun kütle, atalet gibi fiziksel özelliklere sahip sert bir parçasıdır (örn. bir kol, bir tekerlek).
- `<inertial>` (**Atalet**): Linkin dinamik özelliklerini tanımlar: `<mass>` (kütle) ve `<inertia>` (dönme eylemsizlik matrisi - I_{xx} , I_{yy} , I_{zz}).
- `<collision>` (**Çarpışma**): Fizik motoru (Bölüm 4) tarafından çarpışma tespiti için kullanılan geometriyi tanımlar. Performans için genellikle basit şekiller (primitive) veya düşük poligonlu (low-polygon) mesh'ler kullanılır.
- `<visual>` (**Görsel**): Render motoru (Bölüm 2) tarafından görselleştirme için kullanılan geometriyi tanımlar. Genellikle yüksek poligonlu, dokulu ve gerçekçi mesh'ler kullanılır.
- `<joint>` (**Ekleme**): İki `<link>` 'i birbirine bağlar. Bir `parent` (ebeveyn) ve `child` (çocuk) link tanımlar ve aralarındaki hareketin türünü (örn. `revolute` - döner, `prismatic` - kayar) ve eksenini (`<axis>`) kısıtlar.

3.4 Geometri Türleri ve Mesh İçe Aktarma

`<collision>` ve `<visual>` etiketleri içinde üç tür geometri tanımlanabilir:

1. **Basit Şekiller:** `<box>`, `<sphere>`, `<cylinder>`. Hesaplaması en hızlı ve en kararlı olanlardır. Çarpışma geometrileri için **şiddetle** tavsiye edilirler.
2. **Mesh (Ağ):** `<mesh>` etiketi ile karmaşık 3D modellerin (mesh) kullanılmasını sağlar.
 - **Desteklenen Formatlar:** COLLADA (.dae), STL (.stl) ve OBJ (.obj).
 - **Kullanım:** Mesh'ler, `<uri>` etiketiyle dosya yolunu belirterek içe aktarılır. Örneğin, bir robotun şasisi, çarpışma için basit bir `<box>` geometrisi kullanırken, görselleştirme için karmaşık bir `chassis.dae` mesh'i kullanabilir.
3. **Heightmap:** `<heightmap>` (Bölüm 3.2'de açıklandığı gibi), araziler için kullanılır.

Vaka Çalışması: İki Tekerlekli Robotun İnşası

Sıfırdan bir robot modellemek, bu bileşenlerin nasıl bir araya geldiğini gösterir:

1. **Yapı:** `~/gazebo/models/my_robot` dizini oluşturulur. İçine `model.config` (meta veri) ve `model.sdf` (model tanımı) dosyaları eklenir.
2. **Şasi (Chassis):** İlk olarak, bir `<link name='chassis'>` oluşturulur. Bu linkin hem `<collision>` hem de `<visual>` etiketleri içinde basit bir `<box>` geometrisi tanımlanır.
3. **Tekerlekler (Wheels):** İki yeni `<link name='left_wheel'>` ve `<link name='right_wheel'>` eklenir. Bunlar için `<cylinder>` geometrisi kullanılır. Pozları, şasinin yanlarında doğru konumda olmaları için ayarlanır.
4. **Caster Teker:** Şasiye bağlı (aynı link içinde) bir denge tekeri olarak sürtünmesiz bir `<sphere>` geometrisi eklenir.
5. **Eklemler (Joints):** Model, `<static>false</static>` olarak ayarlanarak dinamik hale getirilir. Ardından, tekerleklerin dönmesini sağlamak için iki adet `<joint type='revolute'>` (döner eklemler) eklenir. Bu eklemler `chassis` linkini (parent) `left_wheel` ve `right_wheel` linklerine (child) bağlar ve dönme eksenini (`<axis>`) tanımlar.

3.5 Hiyerarşik Yapılar ve Programatik Animasyon

- **İç İçe Modeller (Nested Models):** SDF 1.5 (Gazebo 7+) ve üzeri sürümlerde, bir `<model>` etiketi başka bir `<model>` etiketini içerebilir. Bu, karmaşık montajların (örn. bir robot koluna takılı bir gripper) modüller olarak oluşturulmasına olanak tanır. İç içe geçmiş bir modelin linkine (`model_01::link_01`) eklemlerle bağlanılabilir.
- **Programatik Animasyon:** Aktörler (Bölüm 2.3) fizik dışı animasyon sağlarken, modellerin kendileri de eklentiler (Bölüm 6) aracılığıyla programatik olarak canlandırılabilir. "Animated Box" örneği bunun kanıtıdır:
 - `animated_box.cc` : Simülasyonun her döngüsünde kutunun pozunu güncelleyen bir Model Eklentisidir.
 - `independent_listener.cc` : Kutunun pozunu yayınladığı Gazebo Transport topic'ine (Bölüm 1.3) abone olan ve veriyi okuyan ayrı bir programdır.

BÖLÜM 4: FİZİK SİMÜLASYONU VE MOTORLAR

4.1 Çoklu Fizik Motoru Desteği

Gazebo Classic 11, tek bir fizik motoruna bağlı değildir. Bunun yerine, farklı motorları "tak-çıkart" şeklinde

kullanmaya izin veren soyut bir fizik arayüz katmanına sahiptir. Desteklenen motorlar şunlardır:

- **ODE (Open Dynamics Engine):** Varsayılan motor. Hızlı, kararlı ve robotik için optimize edilmiştir.
- **Bullet:** Özellikle çarpışma tespiti konusunda güçlü olan popüler bir açık kaynaklı motordur.
- **Simbody:** Biyomekanik ve karmaşık mafsallı sistemler için tasarlanmış bir motordur.
- **DART (Dynamic Animation and Robotics Toolkit):** Karmaşık temaslara ve esnek gövdelere odaklanan modern bir motordur.

Kullanılacak motor, dünya SDF dosyasındaki `<physics>` etiketi içindeki `type` parametresi (örn. `<physics type='ode'>`) ile seçilir.

4.2 Open Dynamics Engine (ODE) Parametreleri

Simülasyonun kararlılığı, hızı ve gerçekçiliği, doğrudan dünya SDF dosyasındaki `<physics>` etiketi altında ayarlanan parametrelere bağlıdır. Varsayılan motor ODE olduğu için, bu parametrelerin "incik cıncığına" hakim olmak, Gazebo'da uzmanlaşmanın anahtarıdır.

Aşağıdaki tablo, en kritik ODE parametrelerini ve işlevlerini özetlemektedir:

Parametre	SDF Yolu (ODE altında)	Varsayılan	Açıklama ve "Nasıl Çalışır"
Max Step Size	<code><physics max_step_size></code>	0.01	Simülasyonun her bir fizik adımının saniye cinsinden süresi. Düşürmek doğruluğu artırır ancak performansı düşürür.
Update Rate	<code><physics real_time_update_rate></code>	1000	Simülasyonun saniyede kaç adım atmaya çalışacağı (Hz). <code>Update Rate * Step Size = 1.0</code> (varsayılan) gerçek zamanlı faktörü hedefler.
Solver Type	<code><solver type></code>	quick	ODE çözücüsünün türü. <code>world</code> (Dantzig) daha doğrudur ancak yavaştır. <code>quick</code> (iterative PGS) daha hızlıdır ancak doğruluk için <code>iters</code> 'a bağlıdır.
Iterations	<code><solver iters></code>	50	<code>quick</code> çözücünün her adımda yapacağı iterasyon sayısı. Artırmak, eklemelerin ve temaların kararlılığını/doğruluğunu artırır, ancak yavaşlatır.
SOR	<code><solver sor></code>	~1.3	Başarılı Aşırı Gevşeme (Successive Over-Relaxation) parametresi. <code>quick</code> çözücünün yakınsamasını hızlandırmaya yardımcı olabilir.
CFM	<code><constraint s cfm></code>	0.0	(Kritik Parametre) Kısıtlama Kuvveti Karıştırma (Constraint Force Mixing). Simülasyon "patladığında" ayarlanacak ilk parametredir. O'dan büyük bir değer (örn. $1e-5$), sert temaları "yumuşatarak" (bir yay gibi davranmasını sağlayarak) sistemi kararlı hale getirir.
ERP	<code><constraint s erp></code>	0.2	(Kritik Parametre) Hata Azaltma Parametresi (Error Reduction Parameter). Her adımda eklem ve temas hatalarının (örn. penetrasyon) ne kadarının düzeltileceğini belirler. <code>cfm</code> ile birlikte temaların "sertliğini" tanımlar.
Max Correcting Vel	<code><constraint s contact_max_correcting_vel></code>	100	Penetrasyon hatalarını düzeltirken izin verilen maksimum hız. Düşük değerler, "patlamaları" önler.

Surface Layer	<code><constraint s contact_surface_layer></code>	0.001	Temasların hangi penetrasyon derinliğinde "aktif" hale geleceğini belirler.
Max Contacts	<code><physics max_contacts></code>	20	İki varlık arasında izin verilen maksimum temas noktası sayısı. Karmaşık mesh'ler (ağlar) için performansı artırmak amacıyla düşürülebilir.

4.3 Temas, Sürtünme ve Çarpışma Yönetimi

4.3.1 Sürtünme (Friction)

ODE'deki varsayılan sürtünme modeli, Coulomb sürtünmesini iki yönlü bir yaklaşımla ele alır:

- `mu` : Birincil sürtünme yönü (genellikle hareket yönü) için katsayı.
- `mu2` : Birincil yöne dik olan ikincil yön için katsayı.
- Seçim Mekanizması: İki nesne çarpıştığında, Gazebo temas için her iki nesnenin `mu` ve `mu2` değerlerinin en küçüğünü seçer.
- Sürtünme Modelleri: ODE, SDF'de `<friction_model>` etiketiyle seçilebilen farklı modeller sunar:
 - `pyramid_model` (Varsayılan): İki yönü bağımsız olarak ele alır.
 - `box_model` : Normal kuvvetten bağımsız, sabit bir sürtünme kuvveti limiti uygular.
 - `cone_model` : Daha fiziksel olarak gerçekçi, birleşik bir sürtünme konisi modeli kullanır (sadece `mu` kullanılır).

4.3.2 Torsional Sürtünme (Dönme Sürtünmesi)

Standart Coulomb sürtünmesi, bir bilyenin veya topun dönmesini (spin) durdurmak için yetersizdir, çünkü temas noktası çok küçüktür. Bu gerçekçi olmayan "sonsuz dönme" sorununu çözmek için Gazebo, sadece ODE motorunda çalışan torsional sürtünmeyi destekler.

Bu sürtünme, dönme hareketine karşı bir tork uygular ve temas alanının yarıçapına bağlıdır. SDF'de `<friction><torsional>` altında yapılandırılır:

- `coefficient` : Torsional sürtünme katsayısı.
- `use_patch_radius` : `true` ise `patch_radius` kullanılır, `false` ise `surface_radius` kullanılır.
- `patch_radius` / `surface_radius` : Temas alanının yarıçapını tanımlar.

Kritik Not: Varsayılan olarak `patch_radius` ve `surface_radius` sıfırdır (0). Bu, torsional sürtünmenin varsayılan olarak kapalı olduğu ve bir topun veya tekerleğin realistik bir şekilde yavaşlaması isteniyorsa manuel olarak bu parametrelerin ayarlanması gerektiği anlamına gelir.

4.3.3 Collide Bitmasks (Çarpışma Bit Maskeleri)

Gazebo, hangi nesnelerin birbiriyle çarpışıp çarpışmayacağını kontrol etmek için bit maskeleme adı verilen güçlü bir mekanizma sağlar. Bu, hem performans optimizasyonu (gereksiz kontrolleri atlamak) hem de mantıksal ayırma (örn. bir robotun kendi parçalarıyla veya belirli nesnelerle çarpışmasını engellemek) için kullanılır.

Nasıl Çalışır:

1. Her çarpışma geometrisine SDF içinde bir `<collide_bitmask>` (16-bitlik bir sayı, genellikle onaltılık - hexadecimal - olarak verilir, örn. `0x01`) atanır.
2. Varsayılan maske `0xFFFF` 'tir (yani "her şeyle çarpış").
3. İki nesne (A ve B) karşılaştığında, fizik motoru bit düzeyinde AND (&) işlemi uygular: `(A_mask & B_mask)` .
4. Eğer sonuç sıfır (0) ise, çarpışma **yok sayılır**. Eğer sonuç sıfırdan büyükse, çarpışma hesaplanır.

Örnek: Bir robotun Lidar'ı (mask = `0x01`) sadece duvarlarla (mask = `0x02`) değil, aynı zamanda diğer robotlarla (mask = `0x04`) da çarpışsın, ancak zeminle (mask = `0x08`) çarpışmasını istiyorsak; Lidar'a `0x07` (`0x01` | `0x02` | `0x04`) gibi bir maske verilebilir.

BÖLÜM 5: SENSÖR MODELLEME VE VERİ ÜRETİMİ

Gazebo'nun temel güçlerinden biri, çok çeşitli sensörleri yüksek doğrulukla simüle edebilmesidir.

Sensörler, bir `<link>` 'e bağlı `<sensor>` etiketleri olarak SDF içinde tanımlanır ve genellikle verilerini Gazebo veya ROS topic'leri üzerinden yayınlırlar.

Gazebo, mükemmel ("ground truth") veriden, gerçekçi gürültü ve bozulmalara sahip verilere kadar bir "gerçekçilik spektrumu" sunar. Bu, bir robotik sisteminin algılama, durum tahmini veya planlama gibi farklı alt bileşenlerini izole olarak test etmek için kritik öneme sahiptir.

5.1 Desteklenen Sensör Tipleri ve Gürültü Modelleri

5.1.1 Ray/Lidar (Işın) Sensörleri

- **Nasıl Çalışır:** Simüle edilmiş lazer ışınları gönderir ve çevredeki nesnelerle olan mesafeyi (menzili) ölçer (örn. Hokuyo, Velodyne).
- **Gürültü Modeli:** Gürültü, her bir ışının menziline (range) eklenir. SDF'de `<noise><type>gaussian</type></noise>` ile yapılandırılır.
 - `mean` (ortalama) ve `stddev` (standart sapma) metre cinsinden tanımlanır.
 - Gürültü eklendikten sonra, sonuç sensörün minimum (`<min>`) ve maksimum (`<max>`) menzil değerleri arasında kalacak şekilde "kelepçelenir" (clamped).

5.1.2 Kamera Sensörleri (RGB, Derinlik)

- **Nasıl Çalışır:** OGRE render motorunu (Bölüm 2.1) kullanarak 2D görüntü (RGB) veya derinlik haritası (Depth Camera) verisi üretir.
- **Gürültü Modeli:** Çıkış amplifikatör gürültüsünü (output amplifier noise) modeller. Her bir pikselin her bir renk kanalına ([0.0, 1.0] aralığında) **bağımsız** olarak Gaussian gürültü eklenir.
 - **Kritik Not:** Bu gürültü modeli bir GLSL shader (grafik kartı programı) içinde uygulanır ve **çalışmak için bir GPU gerektirir**.
- **Lens Distorsiyonu (Bozulma):** Gerçekçi lens bozulmalarını simüle etmek için Brown's distorsiyon modelini destekler. SDF'de `<distortion>` etiketi altında şu katsayılar tanımlanır:
 - Radyal Bozulma: `k1` , `k2` , `k3`
 - Teğetsel Bozulma: `p1` , `p2`
 - `center` : Görüntü merkezi (örn. 0.5 0.5). Bu, özellikle kamera kalibrasyonu ve SLAM/Görsel Odometri algoritmalarını test etmek için hayati öneme sahiptir.

5.1.3 IMU (Ataletsel Ölçüm Birimi) Sensörleri

- **Nasıl Çalışır:** Açısal hızları (angular rates) ve çizgisel ivmeleri (linear accelerations) ölçer.
- **Gürültü Modeli:** Gazebo'daki en karmaşık ve gerçekçi gürültü modellerinden birine sahiptir. Oryantasyon (quaternion) verisine gürültü uygulanmaz, sadece ham ivme ve hız verilerine uygulanır. İki tür bozulma vardır:
 1. **Noise (Gürültü):** Her bir ölçüme (X, Y, Z eksenleri için ayrı ayrı) eklenen, `mean` ve `stddev` ile tanımlanan anlık Gaussian gürültü.
 2. **Bias (Sapma/Önyargı):** Gerçek IMU'lardaki "drift" (sürüklenme) hatasını simüle eder. Bu, simülasyonun başında bir kez `bias_mean` ve `bias_stddev` ile örneklenen ve ardından simülasyon boyunca sabit kalan bir ofset değeridir.

5.1.4 Temas (Contact) Sensörü

- **Nasıl Çalışır:** SDF'de belirtilen bir çarpışma geometrisine (`<contact><collision>...`) temas eden diğer nesneleri raporlar.
- **Veri Alma Yöntemleri:**
 1. **Doğrudan Topic:** Sensör, `msgs::Contacts` türünde bir Gazebo topic'i (Bölüm 1.3) üzerinden temas verilerini (çarpışan cisimlerin adları, temas noktası, normali ve derinliği) yayınlar.
 2. **Sensor Plugin:** Bir C++ Sensor Plugin (Bölüm 6) yazılarak, `OnUpdate` callback'i aracılığıyla temas verilerine programatik olarak doğrudan erişilebilir.

5.1.5 Kuvvet/Tork (Force/Torque) Sensörü

- **Nasıl Çalışır:** Bir eklem (joint) üzerine etkiyen 3 eksenli kuvvetleri ve 3 eksenli torkları ölçer.
- **Kritik Not:** Bu sensör bir `<link>` 'e değil, bir `<joint>` etiketinin içine yerleştirilmelidir.
- **Parametreler:**
 - `<frame>` : Ölçümlerin hangi referans çerçevesinde ifade edileceğini belirler (seçenekler: `child` , `parent` , `sensor`).
 - `<measure_direction>` : Kuvvetin ölçüm yönünü belirler (örn. `child_to_parent` - çocuk linkten ebeveyn linke uygulanan kuvvet).

5.1.6 Logical Camera Sensörü (Gazebo 7.0+)

- **Nasıl Çalışır:** Bu, algılama (perception) katmanını *atlamak* için kullanılan özel bir sensördür. Geleneksel bir kamera gibi piksel verisi *üretmez*. Bunun yerine, görüş alanı (frustum) içindeki modellerin bir listesini ve bu modellerin kamera çerçevesine göre **pozlarını** (pose) raporlar.
- **Kullanım Alanı:** Nesne tespiti veya SLAM algoritmalarını test etmek yerine, sadece "nesnenin pozunu mükemmel olarak bildiğimizi varsayarak" görev planlama (task planning) veya manipülasyon algoritmalarını (örn. "kutuyu bul ve al") test etmek için kullanılır.

BÖLÜM 6: EKLENTİ (PLUGIN) SİSTEMİ İLE PROGRAMATİK KONTROL

Gazebo'nun belki de en güçlü özelliği, çekirdek kodunu değiştirmeden simülasyonun hemen her yönünü kontrol etmeye ve genişletmeye olanak tanıyan C++ eklenti (plugin) sistemidir. Eklentiler, simülasyon dünyasına, modellere, sensörlere veya arayüze bağlanan, dinamik olarak yüklenen paylaşımlı kütüphanelerdir (`.so` dosyaları).

6.1 Eklenti Felsefesi ve C++ API Geliştirme Süreci

Bir eklenti yazmanın temel adımları "Hello World" ve "Model Push" örneklerinde özetlenmiştir:

1. **Kod Yazma:** İlgili eklenti türünden (örn. `WorldPlugin`, `ModelPlugin`) miras alan bir C++ sınıfı oluşturulur.
2. **Load Fonksiyonu:** Eklenti yüklendiğinde Gazebo tarafından çağrılan ana giriş noktasıdır. Bu fonksiyon, simülasyondaki ilgili parçaya (dünya, model, sensör) bir işaretçi (pointer) ve SDF'deki eklenti parametrelerine (`sdf::ElementPtr`) erişim sağlar.
3. **Olay Bağlantısı (Event Connection):** Eklentinin simülasyon döngüsüyle etkileşime girmesi gerekir. Bu genellikle `Load` fonksiyonu içinde, `OnUpdate` gibi özel bir fonksiyonun `event::Events::ConnectWorldUpdateBegin` (her simülasyon adımının başında çağrılır) gibi bir olaya bağlanmasıyla gerçekleştirilir.
4. **Kayıt (Registration):** Eklenti sınıfı, `GZ_REGISTER_WORLD_PLUGIN(MyPlugin)` veya `GZ_REGISTER_MODEL_PLUGIN(MyPlugin)` gibi bir makro kullanılarak simülatöre tanıtılır.
5. **Derleme:** `CMakeLists.txt` kullanılarak bir paylaşımlı kütüphane (`.so` dosyası) oluşturulur.
6. **Kullanım:** Oluşturulan `.so` dosyasının bulunduğu dizin `GAZEBO_PLUGIN_PATH` ortam değişkenine eklenir. Son olarak, eklenti, SDF dosyası içinden `<plugin name="..." filename="libmy_plugin.so"/>` etiketi ile çağrılır.

6.2 Eklenti Tipleri ve Vaka Çalışmaları

Gazebo'da her biri farklı bir kapsama ve amaca hizmet eden birden fazla eklenti türü bulunur:

World Plugin (Dünya Eklentisi)

- **Kapsam:** Tüm simülasyon dünyasını (world) kontrol eder.
- **Yetkileri:** Dünyanın fizik özelliklerini (yerçekimi gibi) değiştirebilir, simülasyona modeller ekleyip çıkarabilir (fabrika işlevi).
- **Vaka Çalışması 1:** `world_edit.cc` : Bu eklenti, `Load` fonksiyonu içinde Gazebo Transport (Bölüm 1.3) üzerinden `~/physics` topic'ine bir `msgs::Physics` mesajı yayınlarak yerçekimini programatik olarak değiştirir.
- **Vaka Çalışması 2:** `factory.cc` : Bu eklenti, dünyaya model eklemenin iki yolunu gösterir: (1) Doğrudan `_parent->InsertModelFile()` (world pointer üzerinden) fonksiyonunu çağırarak veya (2) `~/factory` topic'ine yayınlanan `msgs::Factory` mesajlarını dinleyerek.

Model Plugin (Model Eklentisi)

- **Kapsam:** Belirli bir modele (örn. bir robot) bağlanır ve o modelin linklerini ve eklemlerini kontrol eder.
- **Vaka Çalışması:** `model_push.cc` : Bu eklenti, "Animated Box" örneğinin arkasındaki mekanizmadır. `Load` içinde `ConnectWorldUpdateBegin` olayına bağlanır. Tetiklenen `OnUpdate` fonksiyonu içinde `this->model->SetLinearVel(...)` çağrısı yaparak modele (örn. bir kutuya) sürekli bir itme kuvveti uygular.

Sensor Plugin (Sensör Eklentisi)

- **Kapsam:** Belirli bir sensör örneğine bağlanır. Ham sensör verisine erişmek, onu işlemek veya manipüle etmek için kullanılır.
- **Vaka Çalışması:** `ContactPlugin.cc` : `Load` fonksiyonunda sensör işaretçisini (`parentSensor`) alır ve `parentSensor->ConnectUpdated()` ile `OnUpdate` fonksiyonunu sensörün güncelleme olayına

bağlar. `OnUpdate` içinde `parentSensor->Contacts()` çağrısı yaparak temas verilerini okur ve konsola yazdırır.

GUI Plugin (Grafik Arayüz Eklentisi)

- **Kapsam:** `gzclient` 'in (istemci) arayüzünü genişletir. Render penceresinin üzerine özel QT widget'ları (butonlar, kaydırıcılar) ekler.
- **Vaka Çalışması:** `GUIExampleSpawnWidget` : `GUIPlugin` sınıfından miras alır. QT kullanarak bir "Spawn Sphere" (Küre Oluştur) butonu oluşturur. Butona tıklandığında, bir `msgs::Factory` Protobuf mesajı oluşturur ve bunu `~/factory` topic'ine yayınlar.

System Plugin (Sistem Eklentisi)

- **Kapsam:** `gzserver` veya `gzclient` başladığında yüklenen ve simülasyonun veya arayüzün çekirdek işleyişine bağlanan eklentiler.
- **Vaka Çalışması:** `system_gui.cc` : `gzclient` için bir sistem eklentisidir. `event::Events::ConnectPreRender` olayına (yani her bir kare render edilmeden hemen önce) bağlanır ve aktif kameradan (`gui::get_active_camera()`) görüntüleri diske kaydeder.

Bu eklenti sistemi, Gazebo'nun mimari bütünlüğünün mükemmel bir yansımasıdır.

`GUIExampleSpawnWidget` vaka çalışması, tüm mimariyi özetler: `gzclient` 'ta çalışan bir **GUI Eklentisi**, bir butona basıldığında bir Protobuf mesajını **Gazebo Transport** katmanına yayınlar. `gzserver` 'da çalışan bir **World Eklentisi** bu mesaja abonedir, onu alır ve modeli fizik dünyasında oluşturur. Bu, istemci -> taşıma -> sunucu döngüsünün eksiksiz bir gösterimidir.

BÖLÜM 7: HARİCİ ENTEGRASYONLAR VE YÖNETİM

Gazebo, tek başına bir simülatör olmanın ötesinde, daha geniş bir robotik ekosistemiyle, özellikle de ROS ile derinlemesine entegre olacak şekilde tasarlanmıştır.

7.1 Araçlar: Veri Kaydı (Logging) ve Tekrar Oynatma (Playback)

Gazebo, simülasyonları kaydetmek ve deterministik olarak yeniden oynatmak için yerleşik araçlara sahiptir.

- **Kayıt (Logging):**
 - **Nasıl Başlatılır:** GUI'den `Ctrl+D` tuşlarına basarak Veri Kaydedici (Data Logger) arayüzü üzerinden veya `gazebo` 'yu `-r` (`--record`) bayrağıyla başlatarak.
 - **Ne Kaydedilir:** Bir `.log` dosyası. Bu dosya iki tür veri içerir: (1) Simülasyonun 0. saniyesindeki **tam dünya SDF'si** (tüm modellerin, ışıkların ve fiziğin tam tanımı) ve (2) Simülasyon süresince her bir değişiklikte kaydedilen bir dizi "dünya durumu" (world states - modellerin pozları, hızları, eklem durumları vb.).
- **Tekrar Oynatma (Playback):**
 - **Nasıl Başlatılır:** `gazebo -u -p ~/logs/my_log/state.log` komutuyla. `-u` bayrağı simülasyonu duraklatılmış olarak başlatır.
 - `gzclient` özel bir "tekrar oynatma" modunda açılır. Bu modda, zaman çizgisinde geri/ileri sarma, simülasyonu adım adım (step) ilerletme ve belirli bir ana atlama (jump) yapılabilir.

7.2 Robot Operating System (ROS 1) Entegrasyonu: `gazebo_ros_pkgs`

Gazebo Classic 11'in ROS 1 (Noetic/Melodic) ile entegrasyonu, `gazebo_ros_pkgs` adı verilen bir dizi ROS paketi aracılığıyla sağlanır. Bu entegrasyonun "nasıl" çalıştığı, Gazebo'nun eklenti mimarisine dayanır.

Temel Mekanizma: `gazebo_ros_pkgs` , özünde Gazebo'nun dahili **Gazebo Transport** (Protobuf) mesajlarını (Bölüm 1.3) standart **ROS Transport** (ROS msgs) mesajlarına ve servislerine "köprüleyen" bir dizi Gazebo eklentisi içerir.

- `gazebo_ros_api_plugin` : `gzserver` içinde çalışan ana eklentidir. "gazebo" adında bir ROS düğümü (node) başlatır ve Gazebo'nun dahili zamanlayıcısını (simülasyon zamanı) ROS'un geri çağırma (callback) zamanlayıcısıyla entegre eder.
- `gazebo_ros_paths_plugin` : Gazebo'nun `model://` gibi yolları çözerken `package://` gibi ROS paket yollarını da anlamasını sağlar.

ROS Zaman Senkronizasyonu: Bu eklentiler yüklendiğinde, `gazebo_ros_api_plugin` iki kritik şey yapar:

1. ROS Parametre Sunucusuna `/use_sim_time` parametresini `true` olarak ayarlar.
2. Gazebo'nun o anki simülasyon zamanını periyodik olarak `/clock` topic'inde yayınlar.

Bu iki eylem, tüm ROS düğümlerinin zaman kaynağı olarak Gazebo'nun simülasyon zamanını kullanmasını sağlar. Bu, simülasyon duraklatıldığında tüm ROS sisteminin de "duraklaması" anlamına gelir ve deterministik testler için bu hayati önem taşır.

ROS Arayüzleri: Bu entegrasyon, simülasyon üzerinde tam kontrol sağlayan standart ROS servisleri ve topic'leri ortaya çıkarır. Aşağıdaki tablolar en sık kullanılanları özetlemektedir.

Tablo 7.1: `gazebo_ros_pkgs` Tarafından Sağlanan Temel ROS Servisleri

ROS Servisi	Mesaj Tipi	Açıklama
<code>/gazebo/ spawn_urdf_model</code>	<code>gazebo_msgs/ SpawnModel</code>	Bir URDF modelini simülasyona ekler.
<code>/gazebo/ spawn_sdf_model</code>	<code>gazebo_msgs/ SpawnModel</code>	Bir SDF modelini simülasyona ekler.
<code>/gazebo/delete_model</code>	<code>gazebo_msgs/ DeleteModel</code>	Belirtilen modeli simülasyondan siler.
<code>/gazebo/ get_model_state</code>	<code>gazebo_msgs/ GetModelState</code>	Bir modelin pozunu (pose) ve hızını (twist) sorgular.
<code>/gazebo/ set_model_state</code>	<code>gazebo_msgs/ SetModelState</code>	Bir modelin pozunu ve hızını programatik olarak ayarlar.
<code>/gazebo/ apply_body_wrench</code>	<code>gazebo_msgs/ ApplyBodyWrench</code>	Bir linke (body) belirli bir süre boyunca kuvvet (wrench) uygular.
<code>/gazebo/ apply_joint_effort</code>	<code>gazebo_msgs/ ApplyJointEffort</code>	Bir ekleme (joint) belirli bir süre boyunca tork (effort) uygular.
<code>/gazebo/ pause_physics</code>	<code>std_srvs/Empty</code>	Fizik motorunu duraklatır.
<code>/gazebo/ unpause_physics</code>	<code>std_srvs/Empty</code>	Fizik motorunu devam ettirir.

/gazebo/
reset_simulation

std_srvs/Empty

Simülasyonu zaman dahil başa sarar.

Tablo 7.2: gazebo_ros_pkgs Tarafından Yayınlanan Temel ROS Topic'leri

ROS Topic'i	Mesaj Tipi	Açıklama
/clock	rosgaph_msgs/ Clock	Simülasyon zamanını yayımlar (diğer tüm ROS düğümleri için zaman kaynağı).
/gazebo/ model_states	gazebo_msgs/ ModelState	Simülasyondaki tüm modellerin pozlarını ve hızlarını periyodik olarak yayımlar.
/gazebo/ link_states	gazebo_msgs/ LinkStates	Simülasyondaki tüm linklerin pozlarını ve hızlarını yayımlar.

7.3 ROS 1 Aktüatör Yönetimi: ros_control

Robotları ROS üzerinden kontrol etmenin standart yolu ros_control paketini kullanmaktır.

gazebo_ros_pkgs , ros_control ile Gazebo'nun fizik motoru arasında bir arayüz sağlayan özel bir eklenti sunar.

Nasıl Çalışır (Adım Adım):

- URDF'de <transmission> Tanımlama:** Kullanıcı, robotunun URDF dosyasına (veya <gazebo> etiketleri içine) her bir eklem için bir <transmission> etiketi ekler.
- Arayüzü Belirtme:** Bu etiket, bir eklemi (örn. joint1) bir aktüatöre (örn. motor1) bağlar ve kritik olarak, kullanılacak donanım arayüzünü (örn. hardware_interface/EffortJointInterface) belirtir.
- gazebo_ros_control Eklentisi:** Kullanıcı, SDF dosyasına (veya URDF'deki <gazebo> etiketi içine) libgazebo_ros_control.so eklentisini ekler.
- Eklentinin Çalışması:** Bu eklenti yüklendiğinde, URDF'yi okur, tüm <transmission> etiketlerini bulur. Gazebo'nun fizik motorundan (Bölüm 4) bu eklemleri bulur ve EffortJointInterface (veya VelocityJointInterface vb.) gibi ros_control 'ün anladığı standart donanım arayüzlerini oluşturur.
- Kontrolör Yöneticisi:** Eklenti, bu arayüzleri ros_control 'ün ControllerManager 'ına (Denetleyici Yöneticisi) kaydeder.
- Kontrolörleri Başlatma:** Geliştirici, bir YAML dosyasında PID kazançlarını tanımlar ve controller_spawner ROS düğümünü kullanarak (genellikle bir .launch dosyasından) joint_position_controller veya joint_effort_controller gibi standart ros_control denetleyicilerini başlatır.

Sonuç: Bu zincir tamamlandığında, ROS'tan gelen bir komut (örn. "kolu şu açığa götür") ros_control denetleyicisi tarafından alınır, bu denetleyici bir tork komutu hesaplar, bu komut gazebo_ros_control eklentisi aracılığıyla Gazebo'daki simüle edilmiş ekleme bir kuvvet/tork olarak uygulanır ve robot hareket eder.

7.4 JavaScript ve Web Entegrasyonu

Gazebo'nun çekirdek mimarisi (Bölüm 1.3), ROS dışındaki entegrasyonlara da izin verir:

- **GazeboJS (NodeJS):** Gazebo için bir JavaScript istemcisidir. `gzserver` ile aynı Gazebo Transport katmanını kullanarak iletişim kuran bir C++ NodeJs eklentisidir. `require('gazebojs')` ile yüklenir ve `gazebo.publish(...)` ve `gazebo.subscribe(...)` fonksiyonları ile simülasyona (örn. dünyayı duraklatmak) müdahale edilebilir.
- **Gzweb:** Gazebo için bir WebGL (tarayıcı tabanlı) istemcidir. `gzclient` 'in "hafif" bir alternatifidir. Mimarisi, Gazebo Transport ile konuşan bir NodeJS sunucusu, bu sunucudan tarayıcıya veri akışı sağlayan bir WebSocket ve tarayıcıda WebGL ile render yapan bir istemciden oluşur.

BÖLÜM 8: MODERN ALTERNATİF PROFİLİ: ROBOTİK SİMÜLASYON İÇİN UNITY

(Bu bölüm ve takip eden Bölüm 9, *Unity.pdf* dosyasından alınmış ve rapora eklenmiştir.)

Gazebo 11'in kanıtlanmış ancak yaşlanan mimarisine karşılık, robotik topluluğu giderek artan bir şekilde modern oyun motorlarına yönelmektedir. Bu alanda öne çıkan platformlardan biri Unity'dir. Unity, başlangıçta oyun geliştirmek için tasarlanmış olmasına rağmen, fotogerçekçi görselleştirme, güçlü fizik motorları ve genişleyen ekosistemi sayesinde robotik simülasyon için (özellikle algılama ve yapay zeka eğitimi) cazip bir alternatif haline gelmiştir.

8.1 Unity'nin Mimarisi: Oyun Motorundan Simülasyon Platformuna Geçiş

Unity'nin temel mimarisi, Gazebo'nun `gzserver` / `gzclient` ayrımından felsefi olarak farklıdır. Unity, "bileşen tabanlı varlık" (component-based entity) sistemi üzerine kuruludur ve C# programlama dilini kullanır.

Gazebo'da bir robot, tek bir büyük SDF dosyasında tanımlanırken, Unity'de bir robot, "GameObjects" hiyerarşisinden oluşur. Her bir GameObject (örn. bir "robot_kolu_bağlantısı"), Transform (konum/rotasyon), Rigidbody (fizik), MeshRenderer (görsel) ve Collision (çarpışma) gibi bileşenlerin (components) bir koleksiyonudur. Davranışlar (örn. "MotorKontrolcüsü.cs"), bu GameObject'lere eklenen C# script'leri aracılığıyla tanımlanır.

Unity'nin temel yapısı "monolitik" bir editör ve oynatıcı (Player) üzerine kuruludur. Simülasyon geliştirme süreci, Gazebo'nun metin tabanlı SDF ve başsız sunucu yaklaşımından farklı olarak, "Ne Görürsen Onu Alırsın" (WYSIWYG - What You See Is What You Get) prensibine dayanır. Bu, sanatçılar ve tasarımcılar için iş akışını hızlandırırken, Gazebo'nun sunduğu programatik ve başsız (headless) mod esnekliğini (en azından doğal olarak) sunmaz.

8.2 Fizik Simülasyonu: NVIDIA PhysX ve Havok

Unity, Gazebo 11 gibi kullanıcının fizik motorunu seçmesine olanak tanır, ancak seçenekler farklıdır. Unity'nin (standart 3D projeler için) varsayılan fizik motoru NVIDIA PhysX'tir. Alternatif olarak (özellikle DOTS mimarisiyle) Havok da kullanılabilir.

NVIDIA PhysX, öncelikli olarak oyunlarda görsel olarak inandırıcı ve yüksek performanslı fiziksel etkileşimler yaratmak için tasarlanmıştır. Milyonlarca oyuncunun kullandığı cihazlarda yüksek kare hızlarını (framerate) korumak için optimize edilmiştir.

Bu tasarım felsefesi, robotik simülasyonun temel gereksinimlerinden biri olan determinizm (tekrarlanabilirlik) ile çelişir. PhysX'in varsayılan yapılandırması, performansı korumak için fizik adımlarını birleştirebilir veya küçük hatalara izin verebilir. Bilimsel bir deney veya hassas bir kontrol algoritması doğrulaması için gereken "bit-düzeyinde tekrarlanabilirlik" (bit-wise reproducibility), Unity'de Gazebo/ODE'ye kıyasla elde edilmesi çok daha zor bir hedeftir. Unity'de determinizmi zorlamak (örn. "fixed time

steps" ayarlarını çok hassas kullanmak) mümkündür, ancak bu genellikle platformun performans avantajlarını ortadan kaldırır ve motorun birincil tasarım amacına aykırıdır.

8.3 Görsel Kalite: Yüksek Çözünürlüklü Render Pijaması (HDRP) ve Fotogerçekçilik

Bu, Unity'nin Gazebo 11'e karşı en ezici üstünlüğü sergilediği alandır. Unity, "Scriptable Render Pipelines" (Programlanabilir Render Pijamaları) adı verilen modern bir mimari sunar. Robotik ve otonom sürüş simülasyonları için en alakalı olanı High Definition Render Pipeline (HDRP)'dir.

HDRP, Gazebo 11'in OGRE motorunun yeteneklerinin fersah fersah ötesinde olan fotogerçekçi görüntüleme için tasarlanmıştır. Temel teknolojileri şunları içerir:

- **PBR (Physically Based Rendering - Fiziksel Tabanlı Görselleştirme):** Materyaller (örn. metal, plastik, cam, ıslak asfalt) ışıkla fiziksel olarak doğru bir şekilde etkileşime girer. Bu, bir kameranın "gördüğü" şeyin, sadece şekil değil, aynı zamanda malzemenin dokusu ve yansıtıcılığı olduğu algılama algoritmaları için kritiktir.
- **Gelişmiş Aydınlatma ve Gölgeleme:** Global Illumination (GI - Küresel Aydınlatma), yumuşak gölgeler, ortam tıkanıklığı (ambient occlusion) ve gerçek zamanlı yansımalar (reflections).
- **Gerçek Zamanlı Işın İzleme (Ray Tracing):** Bu, Gazebo'nun CPU tabanlı ışın atmasından (ray-casting) farklıdır. Bu, ışığın fiziksel davranışını (yansımalar, kırılmalar) simüle ederek görsel amaçlı (fotogerçekçi gölgeler ve yansımalar) kullanılan, GPU hızlandırmalı bir teknolojidir.

Bu fotogerçekçiliğin stratejik önemi, "güzel görünmekten" çok daha derindir. Modern robotik algoritmalarının çoğu, özellikle Evrişimli Sinir Ağları (CNN'ler) ve derin öğrenme tabanlı algılama sistemleri, görsel verilere dayanır. Bu CNN'ler, aydınlatma koşullarındaki, gölgelerdeki, yansımadaki ve dokulardaki küçük değişikliklere karşı son derece hassastır.

Gazebo 11'in OGRE motoruyla eğitilen bir CNN, "sim-to-real" açığı (simülasyon ile gerçek dünya arasındaki fark) nedeniyle gerçek dünyada başarısız olacaktır. Unity'nin HDRP'si, bu görsel nüansları doğru bir şekilde taklit ederek "sim-to-real" açığını görsel sensörler (kameralar) için önemli ölçüde azaltır. Bu, Unity'yi otonom sürüş veya görsel denetim sistemlerini eğitmek için çok daha üstün bir platform haline getirir.

8.4 Unity Robotik Ekosistemi

Unity, robotik topluluğunun ihtiyaçlarını karşılamak için (Unity Technologies'in kurumsal desteğiyle) bir dizi özel araç ve paket geliştirmiştir:

- **ROS-TCP-Connector (S_UNITY_ROS):** Bu, Unity'nin ROS ile birincil entegrasyon yöntemidir. Gazebo'nun `gazebo_ros_pkgs` yaklaşımından (aynı süreçte çalışan C++ eklentisi) mimari olarak tamamen farklıdır. ROS-TCP-Connector, Unity'nin C# dünyası ile ROS'un Python/C++ dünyası arasında bir ağ köprüsü kurar. İletişim, TCP üzerinden JSON (veya daha performanslı ikili formatlar) kullanılarak yapılır.
 - **Avantajı:** Muazzam bir esneklik sunar. ROS yığını (master, nodes) bir Docker konteynerinde, Unity simülasyonu Windows bir makinede, kontrol algoritmaları ise başka bir Linux makinesinde çalışabilir.
 - **Dezavantajı:** Ağ gecikmesi (network latency) ve serileştirme yükü (serialization overhead) getirir. Bu, Gazebo'nun paylaşımlı bellek yaklaşımının sağladığı düşük gecikmeli, yüksek frekanslı kontrol döngüleri (Bölüm 3.4) için bir risk oluşturur.
- **URDF Importer:** Standart URDF dosyalarını içe aktarır ve bunları otomatik olarak Unity'nin GameObject hiyerarşisine ve bileşenlerine (Rigidbody, Joint bileşenleri vb.) dönüştürür. Ancak,

Gazebo'nun kullandığı zengin SDF formatını (sensörler, eklentiler) tam olarak desteklemez.

- **Unity ML-Agents (S_UNITY_ML):** Bu, Unity'nin (fotogerçekçilikten sonraki) ikinci büyük stratejik avantajıdır. ML-Agents, doğrudan Unity editörüne entegre olan, yüksek hızlı bir Pekıştirmeli Öğrenme (Reinforcement Learning - RL) arayüzüdür.
 - ML-Agents, Python tabanlı öğrenme kütüphaneleri (örn. TensorFlow, PyTorch) ile Unity simülasyon ortamı (C#) arasında optimize edilmiş bir iletişim kanalı sağlar.
 - Bu, bir ajanın (örn. bir robot kolu) "gözlem yapmasını" (Observe), "karar vermesini" (Decide Action - Python tarafında) ve "hareket etmesini" (Act - C# tarafında) içeren RL döngüsünü son derece yüksek hızlarda (simülasyon zamanını hızlandırarak) gerçekleştirmesini sağlar.
 - Gazebo'da RL eğitimi yapmak mümkündür (genellikle ROS topic'leri üzerinden daha yavaş bir "step" fonksiyonuyla), ancak ML-Agents bu süreci çok daha entegre, verimli ve hızlı hale getirir.

Bu iki temel direğin (HDRP ve ML-Agents) birleşimi, Unity'yi özellikle görsel-tabanlı pekiştirmeli öğrenme (visual reinforcement learning) için (örn. bir dronun sadece kamera kullanarak engellerden kaçınmayı öğrenmesi) Gazebo 11'e kıyasla açık ara üstün bir platform haline getirir. Gazebo 11, Lidar ve IMU kullanarak engelden kaçınmayı eğitmek için verimliyken, Unity, kamera kullanarak aynı görevi eğitmek için çok daha başarılıdır.

BÖLÜM 9: KARŞILAŞTIRMALI TEKNİK ANALİZ VE KIYASLAMA

Gazebo 11.15.1 ve Unity'nin bireysel analizleri (Bölüm 1-8), her iki platformun da farklı felsefeler üzerine kurulu olduğunu göstermiştir. Bu bölüm, kritik robotik metrikleri üzerinden (fizik, sensör, entegrasyon ve performans) bu iki platformu doğrudan karşılaştırmaktadır.

9.1 Fizik Doğruluğu ve Determinizm: ODE (Gazebo) vs. PhysX (Unity)

- **Gazebo 11 (ODE):** Gazebo'nun varsayılan motoru olan ODE, en başından itibaren bilimsel hesaplama ve robotik kontrol (özellikle eklemli sistemler) için tasarlanmıştır. Determinizm (tekrarlanabilirlik) elde etmek için optimize edilebilir. Kullanıcı, çözücü (solver) ayarları (örn. `max_step_size`, iterasyon sayısı) üzerinde tam kontrole sahiptir. Bu, bir PID veya LQR kontrol algoritmasının ince ayarının yapıldığı ve doğrulanmak istendiği senaryolar için hayati öneme sahiptir.
- **Unity (PhysX):** Unity'nin varsayılan motoru olan PhysX, görsel olarak "inandırıcı" ve yüksek performanslı (yüksek kare hızı) oyun deneyimleri için tasarlanmıştır. Varsayılan olarak deterministik değildir. Performans için küçük fiziksel tutarsızlıklara izin verebilir. Determinizmi zorlamak mümkün olsa da (örn. FixedUpdate döngüsünü katı zaman adımlarıyla kullanmak), bu platformun doğal bir gücü değildir ve genellikle Gazebo/ODE'nin sağladığı katı tekrarlanabilirliği garanti etmez.
- **Analiz:** Kontrol algoritmalarının (özellikle yüksek frekanslı ve temas içeren) bilimsel doğrulaması ve tekrarlanabilir testleri için Gazebo 11'in ODE motoru, Unity'nin PhysX motorundan daha güvenilir ve uygun bir zemin sunar.

9.2 Render ve Sensör Gerçekçiliği: OGRE (Gazebo) vs. HDRP (Unity)

Bu, iki platform arasındaki en keskin ve en stratejik farktır.

- **Gazebo 11 (OGRE):** Gazebo'nun OGRE motoru, "fonksiyonel gerçekçilik" sunar. Geometriyi (Lidar için önemlidir) ve temel kamera görüntülerini (bozulma modelleriyle) doğru temsil eder. Ancak, aydınlatma, gölgeler ve materyal özellikleri (PBR) basittir. Bu, modern CNN tabanlı algılama algoritmaları için "sim-to-real" açığını çok geniş bırakır.

- **Unity (HDRP):** Unity'nin HDRP motoru, "fotogerçekçilik" sunar. Aydınlatma (GI, yansımalar), gölgeler ve en önemlisi fiziksel tabanlı materyalleri (PBR) doğru bir şekilde simüle eder. Bu, kamera görüntülerinin gerçek dünyadaki koşullara (farklı hava durumları, günün saatleri, malzeme yansımaları) çok daha yakın olmasını sağlar.
- **Analiz:** Proje hedefi kontrol/planlama (Lidar/IMU/GPS tabanlı) ise Gazebo 11 yeterlidir ve OGRE'nin basitliği bir performans avantajı bile olabilir. Proje hedefi ML-tabanlı algılama (otonom sürüş, görsel denetim, CNN eğitimi) ise Unity'nin HDRP'si ezici bir üstünlüğe sahiptir ve "sim-to-real" problemini çözmek için tek geçerli seçenek olabilir.

9.3 ROS Entegrasyonu: Derin Entegrasyon (Gazebo) vs. Ağ Köprüsü (Unity)

Her iki platform da ROS ile konuşabilir, ancak bunu tamamen farklı mimarilerle yaparlar.

- **Gazebo 11 (gazebo_ros_pkgs):** Entegrasyon, `gzserver` işlemi içinde çalışan C++ eklentileri (plugins) aracılığıyla sağlanır. Veri aktarımı, paylaşımlı bellek (shared memory) veya en azından çok verimli bir süreç içi (in-process) çağrı ile gerçekleşir. Bu, son derece düşük gecikmeli ve yüksek bant genişlikli bir iletişim sağlar. Robotikçiler için standart, "yerel" (native) hissettiren bir iş akışıdır. Dezavantajı, Gazebo'yu ROS'a (ve özellikle ROS Noetic gibi belirli bir sürüme) sıkıca bağlamasıdır.
- **Unity (ROS-TCP-Connector):** Entegrasyon, bir ağ köprüsü (network bridge) üzerinden yapılır. Veriler TCP üzerinden (genellikle JSON olarak) serileştirilir ve ağ üzerinden gönderilir. Bu, muazzam bir esneklik sağlar (dağıtık sistemler, Docker, farklı işletim sistemleri). Ancak, bu mimari, ağ gecikmesi (network jitter) ve serileştirme/deserileştirme yükü getirir.
- **Analiz:** Yüksek frekanslı (>100Hz) "hard real-time" benzeri kontrol döngüleri (örn. bir bacaklı robotun denge kontrolcüsü) için Gazebo'nun `gazebo_ros_pkgs` mimarisi teknik olarak üstündür. Daha yavaş algılama verisi (örn. 10Hz kamera) veya daha yüksek seviyeli komutlar (navigasyon hedefleri) için Unity'nin ROS-TCP-Connector esnekliği daha avantajlı olabilir.

9.4 Performans ve Ölçeklenebilirlik: Başsız Mod ve Çoklu Robot

- **Gazebo 11 (Başsız Mod):** `gzserver` / `gzclient` mimarisi sayesinde başsız modu doğal olarak destekler ve bu modda son derece verimlidir. Görselleştirme yükü (OGRE) olmadığında, `gzserver` tüm kaynaklarını fizik ve sensör hesaplamalarına ayırır. Bu, CI/CD sunucularında binlerce test çalıştırmak için idealdir.
- **Gazebo 11 (Çoklu Robot):** Çoklu robot (örn. sürü (swarm)) simülasyonlarını iyi idare eder. Ancak, performans darboğazı genellikle fizik değil, sensörlerdir. (Bölüm 3.2'de analiz edildiği gibi) Lidar gibi sensörlerin CPU tabanlı ışın izlemesi, sensör açısından zengin (örn. Lidar'lı) çok sayıda robot simülasyonunda Gerçek Zaman Faktörü'nü (RTF) hızla düşürür (RTF << 1).
- **Unity (Başsız Mod):** "Batch mode" ile mümkündür, ancak platformun birincil kullanım senaryosu bu değildir ve yapılandırması daha karmaşıktır.
- **Unity (Ölçeklenebilirlik):** Unity'nin modern DOTS (Data-Oriented Technology Stack) mimarisi (eğer kullanılırsa), binlerce basit ajanı simüle etmek için muazzam bir ölçeklenebilirlik sunar (Gazebo'dan çok daha fazla). Standart GameObject mimarisinde ise performans, render yükünün ve fiziğin GPU'ya (PhysX'in GPU hızlandırması ile) kaydırılabilmesi avantajına sahiptir. Sensör verisi üretimi (özellikle kamera) zaten GPU pipeline'ının bir parçasıdır ve Gazebo'nun CPU tabanlı Lidar yaklaşımının aksine, bir performans darboğazı oluşturmaz.

9.5 Ekosistem, Destek ve Gelecek

- **Gazebo 11 (Classic):** Devasa, yıllar içinde oluşmuş bir ROS topluluğuna sahiptir. Milyonlarca satır kod, forum yanıtı ve `gazebo_ros_pkgs` gibi "savaşta test edilmiş" (battle-tested) paketler mevcuttur. Ancak, platformun geleceği yoktur. EOL (End-of-Life) durumundadır. Bu, platformun bittiği anlamına gelir. Yeni geliştirme "Yeni Gazebo" (Ignition) üzerinedir.
- **Unity:** Devasa bir genel geliştirici ekosistemine (oyun, sinema, mimari) sahiptir. Robotik topluluğu daha yenidir ancak hızla büyümekte ve en önemlisi kurumsal olarak (Unity Technologies tarafından) güçlü bir şekilde desteklenmektedir. S_UNITY ML ve S_UNITY_HDRP gibi araçlar sürekli güncellenmekte ve yeni özellikler kazanmaktadır. Ayrıca, Unity'nin (Pro/Enterprise) potansiyel lisanslama maliyetleri, Gazebo'nun tamamen açık kaynak ve ücretsiz doğasıyla tezat oluşturur.

Tablo 9.1: Gazebo 11.15.1 ve Unity (Robotics) Teknik Karşılaştırma Matrisi

(Bu tablo, *Unity.pdf* Bölüm 5'ten alınmıştır.)

Özellik Metriği	Gazebo 11.15.1 (Classic)	Unity (Modern Robotics Hub)	Analiz ve Stratejik Notlar
Birincil Fizik Motoru	ODE (Open Dynamics Engine)	NVIDIA PhysX	ODE, determinizm ve kontrol doğrulaması için; PhysX, görsel hız ve performans için optimize edilmiştir.
Render Motoru	OGRE 3D	High Definition Render Pipeline (HDRP)	Aradaki görsel sadakat farkı (Fonksiyonel vs. Fotogerçekçi), ML tabanlı algılama için en belirleyici faktördür.
ROS Entegrasyon Metodu	<code>gazebo_ros_pkgs</code> (Paylaşımlı Bellek Eklentisi)	ROS-TCP-Connector (Ağ Köprüsü)	Gazebo = Düşük Gecikme, Düşük Esneklik. Unity = Yüksek Gecikme (potansiyel), Yüksek Esneklik.
ML/AI Entegrasyonu	Yok (ROS üzerinden dolaylı ve yavaş)	Unity ML-Agents (Doğrudan, Hızlı Arayüz)	Unity, Pekiştirmeli Öğrenme (RL) eğitimi için açık ara üstündür.
Lidar/ Sensör Modeli	CPU Tabanlı Işın İzleme (Ray-Casting)	GPU Tabanlı (Render Pipeline ile entegre)	Gazebo'nun sensör yükü CPU'yu zorlar (darboğaz); Unity'ninki render pipeline'ını (GPU) kullanır.
Platformun Geleceği	EOL (Ocak 2025), "Classic" serisinin sonu	Aktif Geliştirme, Güçlü Kurumsal Destek	Gazebo 11 bir "miras" (legacy) sistemdir; Unity aktif bir yatırım alanıdır.
Başsız Mod (Headless)	Doğal ve Verimli (<code>gzserver</code> mimarisi)	Mümkün ("Batch Mode"), ancak doğal değil	Gazebo, CI/CD ve bulut tabanlı testler için mimari olarak daha uygundur.
Birincil Kullanım Senaryosu	Kontrol Algoritması Doğrulaması, Lidar/IMU Tabanlı Planlama	Görsel Algılama (CNN) Eğitimi, RL, Fotogerçekçi Simülasyon	
Lisanslama	Tamamen Açık Kaynak (Apache 2.0)	Ücretsiz (Personal) ve Ücretli (Pro/ Enterprise)	Kurumsal kullanım ve destek için Unity'nin bir maliyeti vardır.

BÖLÜM 10: SONUÇ VE STRATEJİK DEĞERLENDİRME (GAZEBO CLASSIC 11)

10.1 Gazebo Classic 11.15.1 Yetenek Özeti

Bu analiz, Gazebo Classic 11.15.1'in son derece modüler, güçlü ve genişletilebilir bir robotik simülasyon platformu olduğunu ortaya koymaktadır. Temel gücü, fizik ve sensör hesaplamalarını (`gzserver`) görselleştirmeden (`gzclient`) ayıran ve bu bileşenler arasında Google Protobufs tabanlı bir taşıma katmanı (Gazebo Transport) kullanan mimarisinden gelmektedir. Bu mimari sayesinde platform, aşağıdakiler de dahil olmak üzere kapsamlı bir yetenek seti sunar:

1. **Çoklu Fizik Motoru:** Başta varsayılan ODE olmak üzere, Bullet, Simbody ve DART gibi farklı fizik motorlarını destekler; `cfm` ve `erp` gibi "incik cıcık" parametrelerle simülasyon kararlılığı üzerinde derin bir kontrol sağlar.
2. **Detaylı Sensör Modelleme:** Standart kameralar ve Lidar'lardan, GPU'da çalışan gürültü modellerine, lens distorsiyonuna ve IMU'lar için gerçekçi "bias" (sapma) simülasyonuna kadar geniş bir yelpaze sunar.
3. **Kapsamlı Eklenti Sistemi:** C++ API'si aracılığıyla, simülasyonun her yönünü (Dünya, Model, Sensör, GUI, Sistem) programatik olarak kontrol etme ve genişletme imkanı tanır.
4. **Güçlü ROS 1 Entegrasyonu:** `gazebo_ros_pkgs` aracılığıyla ROS 1 ekosistemiyle "altın standart" bir entegrasyon sağlar. Özellikle `ros_control` arayüzü, simüle edilmiş aktüatörlerin ROS üzerinden kontrolü için endüstri standardı bir çözüm sunar.

10.2 Kullanım Ömrü Sonu (EOL) ve Gelecek

Gazebo Classic 11.15.1, ROS 1 (Melodic/Noetic) ekosistemi için zirve noktasıdır. Ancak, bu platformun Ocak 2025'te resmi Kullanım Ömrü Sonu'na (EOL) ulaşacağı unutulmamalıdır. Bu rapor, bu son derece başarılı ve olgun platformun yeteneklerini belgeleyen kapsamlı bir teknik referans niteliğindedir. Yeni projeler geliştiren kullanıcıların, bu mimari felsefelerin çoğunu miras alan modern Gazebo (eski adıyla Ignition) platformuna ve ROS 2 ile entegrasyon için `ros_gz` köprüsüne odaklanmaları şiddetle tavsiye edilir.